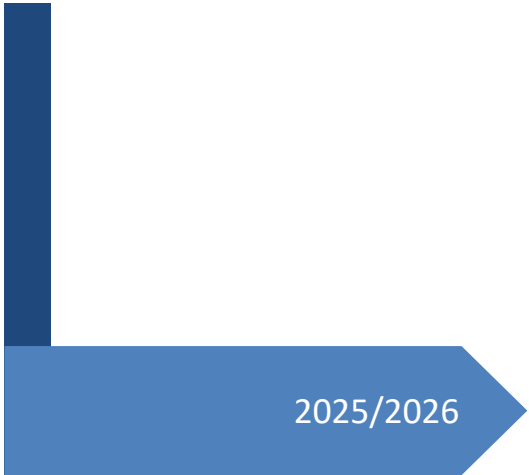
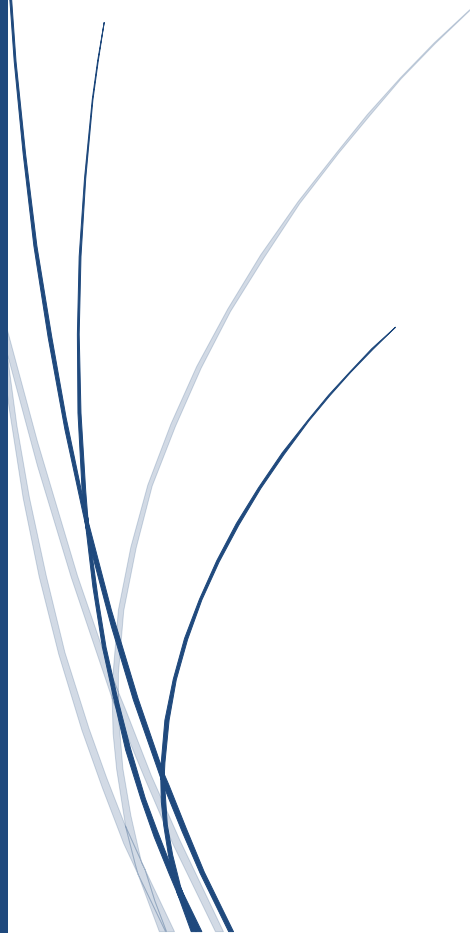




2025/2026

ISW2: Apache Syncope



Matteo Pinfildi
MATRICOLA: 0376255

Sommario

1. Introduzione 1

2. Metodologia 1

 2.1. Milestone 1 1

 2.2 Milestone 2 3

 2.3 Milestone 3 3

 2.4 Milestone 4 4

3. Risultati 4

 3.1 Milestone 1 4

 3.2 Milestone 2 4

 3.3 Milestone 3 5

 3.4 Milestone 4 6

4. Minacce alla validità 8

 4.1 Milestone 1 8

 4.2 Milestone 2 9

 4.3 Milestone 3 9

 4.4 Milestone 4 9

Conclusioni 10

Appendice M1 11

Appendice M2 12

Appendice M3 12

Appendice M4 12

1. Introduzione

Il mantenimento della qualità del software e la prevenzione dei difetti sono sfide centrali nell'ingegneria del software. Questo progetto utilizza come caso di studio il sistema open-source **Apache Syncope**.

Il lavoro è stato strutturato in quattro Milestone, finalizzate a rispondere alle seguenti domande di ricerca:

- **RQ1:** Qual è il miglior classificatore per predire con accuratezza la difettosità delle classi?
- **RQ2:** Come cambia la probabilità di difetto predetta dal miglior classificatore se si simula l'assenza totale di smell?
- **RQ3:** Durante la fase di refactoring automatico con LLM, il modello introduce nuovi code smell durante la correzione? Inoltre, le metriche correlate alla probabilità di difetto aumentano o diminuiscono in seguito all'intervento di refactoring?

Attraverso la risposta a queste domande, la presente relazione illustra i risultati ottenuti in ciascuna fase.

2. Metodologia

2.1. Milestone 1

L'estrazione dei dati storici del progetto Apache Syncope ha avuto inizio con l'identificazione delle release e delle relative date tramite lo script *getReleaseInfo*, il quale ha generato il file *SYNCOPEVersionInfo.csv*. Dall'analisi di questo file sono emerse 79 release totali. Di queste, sono state considerate solo le prime 27, escludendo il restante 66%.

Il dataset finale è strutturato raccogliendo le classi Java del progetto per ciascuna delle release analizzate. Per ogni istanza, è stata condotta una fase di estrazione delle feature. Tale processo si è concentrato sulle seguenti metriche, selezionate per catturare diverse dimensioni della complessità e dell'evoluzione del codice:

- LOC (Lines of Code): misura la dimensione del file in termini di linee di codice;
- LOC_TOUCHED: indica la somma delle linee di codice aggiunte e rimosse nel file all'interno della release analizzata;
- CHURN: indica la somma di tutte le linee di codice aggiunte e rimosse nel file fino alla release analizzata;
- MAX_CHURN: rappresenta il picco massimo di modifiche registrato in un singolo commit effettuato sul file fino alla release analizzata;
- AVC (Average Churn): rappresenta la quantità media di codice modificato nei commit della release analizzata;
- ALA (Average LOC Added): indica il numero medio di linee di codice aggiunte nei commit della release;
- ACSS (Average Change Set Size): misura la dimensione media dei commit, indicando il numero medio di file modificati congiuntamente;
- AGE: indica il tempo trascorso dalla creazione del file;
- AOC (Age of Change): rappresenta il tempo trascorso dall'ultima modifica effettuata sul file;

- NFIX: indica il numero totale di fix registrati nella storia del file fino alla release considerata;
- FIX: indica la quantità di interventi correttivi applicati al file all'interno della singola release analizzata;
- WMC (Weighted Methods per Class): rappresenta la somma della complessità ciclomatica di tutti i metodi definiti nella classe;
- DIT (Depth of Inheritance Tree): indica la profondità della classe all'interno dell'albero di ereditarietà;
- CBO (Coupling Between Objects): misura il livello di accoppiamento tra la classe considerata e le altre classi del sistema;
- RFC (Response For a Class): rappresenta l'insieme dei metodi che possono essere eseguiti in risposta a un messaggio ricevuto dalla classe;
- LCOM (Lack of Cohesion in Methods): misura la mancanza di coesione interna tra i metodi della classe;
- NF (Number of Files): indica il numero massimo di file modificati congiuntamente in un singolo commit all'interno della release analizzata;
- ENTROPY: indica quanto le modifiche apportate a un file sono frammentate su più commit all'interno della release analizzata;
- NR (Number of Revisions): indica il numero totale di commit che hanno interessato il file dall'inizio della sua storia fino alla release analizzata;
- NAUTH (Number of Authors): rappresenta il numero di sviluppatori che hanno apportato modifiche al file fino alla release analizzata;
- NSMELLS: rappresenta il numero di smell rilevati nella classe.

L'estrazione automatizzata delle feature è stata condotta integrando strumenti di analisi statica, quali *CK* per le metriche strutturali e *PMD* per la quantificazione dei Code Smell, con script Java personalizzati per il calcolo delle restanti metriche.

La base per il labeling è stata costruita filtrando esclusivamente i ticket di tipo "*Bug*" con stato "*Closed*" o "*Resolved*" e con risoluzione "*Fixed*". Per ogni ticket sono state identificate la *Opening Version (OV)*, definita come la release corrente al momento dell'apertura della segnalazione, e la *Fixed Version (FV)*, definita come la prima release pubblicata successivamente alla data di risoluzione del ticket. Per determinare l'*Injected Version (IV)*, il sistema analizza innanzitutto il campo *Affected Versions (AV)* del ticket: qualora presente, l'*IV* viene fatta coincidere con la data dell'*AV* più vecchia. Nei casi in cui tale informazione risulti assente, è stato applicato il metodo *Proportion Total*, calcolando una proportion globale che è stata poi usata per stimare l'*IV* sui ticket in cui essa mancava. Il labeling finale è stato eseguito implementando l'algoritmo *SZZ*. I *fix commit* sono stati associati ai ticket *Jira* corrispondenti analizzando i messaggi di commit; da questi sono stati estratti i soli file Java modificati dai fix, ovvero le classi che contenevano il difetto.

Infine, una classe viene etichettata come difettosa (*Bugginess = 1*) in una specifica release se e solo se la data di tale release ricade all'interno del ciclo di vita del bug, ovvero se rispetta la condizione:

$$IV \leq \text{Data_Release} < FV$$

Questo approccio garantisce che il classificatore venga addestrato solo sulle versioni del codice che contenevano effettivamente il bug.

2.2 Milestone 2

La preparazione dei dati per l'addestramento ha richiesto l'eliminazione delle feature puramente identificative, ovvero ID_Release e Class_Name.

Sono stati scelti tre modelli: Ibk, Naive Bayes e Random Forest. Le performance sono state valutate attraverso le seguenti metriche: *Precision*, *Recall*, *AUC* e *Kappa*. Tali metriche non sono state valutate come media pesata, ma misurate specificamente sulla classe di interesse, ovvero l'identificazione dei difetti (*Bugginess* = 1).

Per validare la robustezza dei modelli è stata adottata la tecnica *10 times-10 folds cross-validation*, così da ridurre il rischio di *overfitting*.

Sfruttando l'ambiente Weka, i 3 classificatori sono stati testati incrociando 4 diverse configurazioni di preprocessing (generando 12 iterazioni totali), così da mettere a confronto i valori delle metriche selezionate:

1. No Feature Selection, No Balancing;
2. Si Feature Selection, No Balancing;
3. No Feature Selection, Si Balancing;
4. Si Feature Selection, Si Balancing.

Per la Feature Selection è stato adottato un approccio esplorativo di tipo *Forward greedy*, privilegiato rispetto al *Backward* in quanto restituisce un sottoinsieme di feature dimensionalmente più ridotto e interpretabile. Per il Balancing delle classi è stata implementata una logica di *Undersampling*; tale scelta permette di bilanciare il dataset riducendo la classe di maggioranza (*Bugginess*=0), evitando l'introduzione di rumore o artefatti sintetici tipici delle tecniche di *Oversampling*.

È fondamentale precisare che le operazioni di preprocessing sono state applicate all'interno della *cross-validation*. In questo modo, i filtri hanno agito solo sui dati di training di ogni iterazione, mantenendo il test set completamente isolato e prevenendo qualsiasi forma di *data leakage*.

2.3 Milestone 3

La Milestone 3 si concentra su una *What-If Analysis* per valutare l'impatto dei code smell sui difetti del software.

Sono stati generati quattro dataset distinti:

- Dataset A: è il dataset originale, estratto nella Milestone 1;
- Dataset B+: è una porzione del dataset A che contiene tutte le classi che hanno NSMELLS > 0;
- Dataset B: è una derivazione del dataset B+, ma con una manipolazione artificiale: la metrica NSMELLS viene forzata a 0 per tutte le istanze. Questo scenario simula un "refactoring perfetto" in cui tutti gli smell sono stati rimossi, mantenendo però invariate le altre metriche;
- Dataset C: è una porzione del dataset A che contiene tutte le classi che hanno NSMELLS = 0.

Dopo aver addestrato il *best classifier*, ovvero RandomForest (NO FS, NO Balancing) sul Dataset A, il modello è stato sottoposto ad una fase di inferenza sui quattro dataset (A, B+, B e C). L'obiettivo di questa fase è quello di isolare l'impatto della metrica NSMELLS, così da stimare con precisione quanti difetti del software sarebbero prevenibili in uno scenario ideale privo di code smell.

2.4 Milestone 4

L'obiettivo dell'ultima fase del progetto è quello di valutare l'efficacia e l'affidabilità degli LLM nell'automazione dell'attività di refactoring del codice sorgente. L'esperimento è stato svolto selezionando due classi, una con tanti smell ed una con pochi smell, rispettivamente `DefaultRealmPullResultHandler` e `ResourceLogic`. Per entrambe le classi la versione originale è stata identificata come C0, mentre le varianti generate tramite Copilot sono state identificate come C1, C2, C3 e C4. La generazione delle quattro varianti è avvenuta arricchendo progressivamente il prompt fornito all'LLM. La versione C1 è stata prodotta fornendo all'LLM il codice sorgente originale e richiedendo esclusivamente la rimozione dei code smell rilevati. Per la variante C2, al prompt iniziale sono stati aggiunti i test Black Box. La variante C3 è stata generata integrando sia i test BB, che i test di Control Flow (White Box) derivati dall'analisi con Jacoco. Infine, per la variante C4, il prompt è stato ulteriormente esteso includendo i test Mutation Driven, oltre a quelli BB e CF. È fondamentale precisare che, per tutte le iterazioni in cui è stata fornita una test suite, all'interno del prompt è stato inserito un vincolo esplicito e rigoroso: il codice sottoposto a refactoring doveva obbligatoriamente superare con successo tutti i test forniti.

La valutazione dei risultati è stata condotta confrontando ciascuna variante con l'originale C0. Una prima fase di analisi ha verificato l'effettiva compilabilità del codice e il numero di smell rilevati. Successivamente, l'analisi è stata circoscritta esclusivamente alle seguenti metriche statiche: CBO, LOC, WMC, RFC, LCOM e NSMELLS. In questa fase dell'esperimento è stata calcolata la *correlazione di Spearman* per le metriche selezionate rispetto alla *bugginess* del sistema, così da avere il termine di paragone necessario per interpretare se le variazioni architetturali introdotte dall'LLM rappresentino un effettivo miglioramento o un potenziale rischio per la stabilità del software.

Il prompt utilizzato è visibile in Appendice M4, [Prompt Refactoring](#).

3. Risultati

3.1 Milestone 1

Il dataset finale è composto da 17023 istanze, ognuna delle quali è associata ad una classe Java in una specifica release. Analizzando l'evoluzione temporale del sistema, si osserva una crescita graduale e costante delle sue dimensioni, partendo da 466 classi nella prima release fino a raggiungere le 799 unità nella release 27. Per quanto riguarda la difettosità, i dati evidenziano un forte sbilanciamento: su 17023 istanze totali, le classi "buggy" sono 1145, dunque il 6.73% del totale, mentre le restanti 15878 risultano prive di difetti. L'andamento della difettosità non si mantiene tuttavia lineare nel tempo, registrando un picco nella release 6, in cui 109 classi su 469 presentano difetti, per poi stabilizzarsi, a partire dalla release 20, su valori inferiori al 5%. Parallelamente, l'analisi ha rilevato una diffusa presenza di code smell, infatti, il 62.65% (10665 classi) delle istanze complessive presenta almeno uno smell. Osservando i dati storici, la percentuale di classi affette da smell si rivela stabile per tutte le release prese in esame, partendo dal 65.88% nella prima release e mantenendosi in modo costante tra il 61% e il 66% in tutte le versioni successive. Le tabelle riassuntive sono visibili in Appendice M1, [Tabella 1](#) e [Tabella 2](#).

3.2 Milestone 2

I risultati di questa fase mostrano che la configurazione migliore per il dataset SYNCOPÉ è RandomForest senza balancing e senza feature selection. Questa configurazione ottiene il miglior valore di *AUC*, con 0.92 e il miglior valore di *Kappa*, con 0.61, garantendo il miglior compromesso tra *Precision*, pari a 0.66, e *Recall*, uguale a 0.61. L'unica metrica per cui si registra un valore lievemente superiore in un'altra configurazione è

la *Precision*, che sale a 0.68 nel RandomForest con feature selection ma senza balancing; tuttavia, la scelta della configurazione ottimale ha privilegiato il modello base per non penalizzare la *Recall*, che con la FS scende a 0.59, in quanto nel contesto della defect prediction è molto importante limitare i falsi negativi, ovvero le istanze “buggy” non riconosciute.

Confrontando i restanti classificatori, le prestazioni del modello IBk si collocano esattamente a metà tra gli altri due classificatori. Nello specifico, considerando lo scenario base, il suo valore di AUC, pari a 0.76, si dimostra superiore a quello registrato dal Naive Bayes, pari a 0.71, senza però riuscire a raggiungere l'efficacia del Random Forest. Analizzando i risultati complessivi, il Naive Bayes emerge chiaramente come l'algoritmo meno accurato: in quasi tutti gli scenari testati, infatti, le sue metriche subiscono un netto decremento rispetto alle prestazioni offerte da Random Forest e IBk.

L'analisi delle configurazioni di preprocessing mostra che la feature selection ha un effetto misto a seconda del classificatore utilizzato. Sul RandomForest, come evidenziato dai dati, l'impatto è pressoché neutro, sull'IBk impatta in modo negativo, mentre sul NaiveBayes produce un lieve miglioramento, infatti, la *Precision* passa da 0.19 a 0.21 e il *Kappa* da 0.15 a 0.18.

Ancora più evidente e critico è l'effetto del balancing: l'introduzione della tecnica di bilanciamento produce un effetto complessivamente negativo sulle metriche di RandomForest ed IBK. Ad esempio, applicando il balancing all'IBk, senza FS, si osserva un'apparente crescita della *Recall*, che sale a 0.80, ma si evidenzia un crollo drastico della *Precision*, pari a 0.16, e del *Kappa*, che scende a 0.18. Questa riduzione della *Precision* indica un aumento dei falsi positivi; il modello, pur di intercettare i veri positivi, inizia a classificare erroneamente un numero elevatissimo di istanze “non buggy” come “buggy”. Anche il RandomForest subisce un degrado generale delle prestazioni quando sottoposto a bilanciamento.

In conclusione, come evidenziato all'inizio del paragrafo, il classificatore che predice meglio la *bugginess* delle classi nel dataset SYNCOPE è RandomForest, nella configurazione senza balancing e senza feature selection.

I risultati completi delle dodici configurazioni sperimentali sono riportati in Appendice M2, [Tabella 3](#).

3.3 Milestone 3

I risultati completi di questa fase sono riportati in Appendice M3, [Tabella 4](#).

Analizzando i risultati sui dataset non manipolati, il classificatore RandomForest si dimostra estremamente accurato. Nello specifico, sul Dataset A il modello stima un totale di 1146 istanze difettose; analizzando nel dettaglio le predizioni, si osservano 1143 Veri Positivi (TP) e 3 Falsi Positivi (FP), a fronte di soli 2 Falsi Negativi (FN). Sul Dataset B+ il modello stima 928 istanze difettose, con 926 TP, 2 FP e 2 FN, mentre sul Dataset C ne stima 218, con 217 TP, 1 FP e 0 FN.

Per quanto riguarda l'obiettivo centrale della *What-If Analysis*, ovvero stimare quante classi “buggy” sarebbero potute essere prevenute in assenza di code smell, l'analisi sul Dataset B mostra che l'azzeramento forzato degli smells non ha comportato una diminuzione del volume totale delle predizioni, che è invece lievemente salito da 928 a 939. Analizzando i valori della matrice di confusione, si è evidenziato che l'alterazione della metrica ha generato un aumento di FP, passati dai 2 del Dataset B+ ai 28 del Dataset B;

inoltre, la mancanza dell'informazione sugli smell ha ridotto le predizioni corrette, causando un calo dei TP da 926 a 911, con i FN che salgono da 2 a 17.

Isolando l'impatto sulle singole istanze realmente difettose, si osserva quindi che l'assenza della feature NSMELLS ha portato il modello a non riconoscere più 15 vere classi difettose.

Nel complesso, quindi, la sola eliminazione dei code smell non riduce in modo sostanziale la *bugginess* predetta. Sebbene la presenza di smell si confermi un'informazione utile, da sola non è sufficiente a cambiare le decisioni del modello.

3.4 Milestone 4

I valori della *correlazione di Spearman* sono visibili in [Tabella 5](#). Le correlazioni calcolate tra le metriche considerate e la probabilità di *bugginess* risultano tutte positive e statisticamente significative. In particolare, i valori di correlazione più elevati sono associati a CBO, RFC, LOC e WMC, che evidenziano un legame maggiore con la difettosità rispetto ad NSMELLS. Questo risultato conferma che la valutazione delle varianti prodotte dal refactoring automatico non può limitarsi al solo numero di smell rimossi; al contrario, è necessario monitorare con attenzione l'andamento delle metriche strutturali, poiché interventi focalizzati unicamente sulla risoluzione degli smell potrebbero incrementare le metriche strutturali, risultando controproducenti per la stabilità complessiva del sistema.

La valutazione della classe ResourceLogic ha preso in esame, come punto di partenza, la versione originale C0. L'analisi statica ha rilevato un totale di 4 code smell: due legati alla duplicazione di stringhe costanti, uno relativo alla gestione delle eccezioni e un quarto causato dalla presenza di 9 parametri nel costruttore, un valore che supera la soglia massima di 7 suggerita dalle regole di design.

Nelle varianti C1 e C2 vengono risolti tre dei quattro smell originari. In primo luogo, le stringhe duplicate all'interno del modulo sono state estratte in due costanti private statiche; parallelamente, nel metodo *resolveReference*, la cattura dell'eccezione generica tramite il blocco *catch (Throwable)* è stata sostituita con *catch (Exception)*. In secondo luogo, per affrontare lo smell relativo al costruttore a 9 parametri, l'LLM ha riconosciuto che modificarne la firma avrebbe compromesso la compatibilità con il resto del sistema e, nel caso di C2, avrebbe infranto i vincoli imposti dalle suite di test. Di conseguenza, per non alterare il comportamento della classe, il modello ha preferito aggirare la segnalazione applicando l'annotazione `@SuppressWarnings("java:S107")`.

Nelle varianti C3 e C4, l'aggiunta di test più stringenti, ovvero CF e MT, ha portato l'LLM a tentare ottimizzazioni logiche che si sono rivelate controproducenti sotto il profilo della qualità statica. Nello specifico, nel tentativo di semplificare la logica di un ciclo, il modello ha introdotto troppe istruzioni di *break*, finendo per violare direttamente le regole strutturali di qualità del codice e introducendo un nuovo smell originariamente assente in C0. L'introduzione di questo nuovo smell, unita alla persistenza dello smell causato dal costruttore con più di 7 parametri, ha portato il conteggio totale a 2 smell in entrambe le varianti.

Sulla variante C4, inoltre, l'LLM ha applicato una scomposizione architetturale decisamente più profonda. Analizzando il contesto, il modello ha individuato logiche fortemente ripetute tra i vari moduli della classe. Per rimuovere la ridondanza, ha isolato il codice condiviso all'interno di nuovi metodi privati. Si nota dunque

che C4 ha subito una ristrutturazione invasiva che dovrebbe ottimizzare la modularità complessiva della classe, mantenendone inalterate le funzionalità originali.

Per quantificare gli effetti del refactoring sull'architettura e sulla qualità del codice, sono state estratte le seguenti metriche: CBO, LOC, WMC, RFC e LCOM, per la classe originale e per le quattro varianti. I valori sono visibili in Appendice M4, [Tabella 6](#).

Come si può vedere dai dati, le varianti C1 e C2 mantengono un profilo architetturale quasi identico a C0. L'aumento marginale di 2 della LOC è associabile all'estrazione delle costanti testuali e all'inserimento dell'annotazione `@SuppressWarnings`. Essendo interventi non strutturali, i valori delle metriche WMC, RFC, CBO e LCOM restano inalterati.

Nella variante C3 si osserva una lieve riduzione del WMC, che passa da 37 a 36. Questa variazione, tuttavia, non è un'ottimizzazione, ma la diretta conseguenza della forzatura logica che ha generato un nuovo smell e un incremento di 4 della LOC. RFC, CBO e LCOM restano invece inalterati.

Per quanto riguarda la variante C4 si è osservato un peggioramento nei valori delle metriche: la LCOM passa da 0 a 19, il WMC sale a 39 e l'RFC a 76. Tutto questo a fronte di una riduzione delle righe di codice, con la metrica LOC che scende a 204. Sebbene in C4 l'LLM abbia eliminato le ridondanze, lo ha fatto a discapito dell'architettura; i valori di WMC e RFC, che, in questo caso di studio, figurano tra i più forti predittori di difettosità come dimostrato dalla correlazione di Spearman, non sono migliorati, bensì hanno subito un lieve incremento, accompagnati dal peggioramento della LCOM.

Anche per la classe `DefaultRealmPullResultHandler` si è partiti dall'analisi della classe originale C0. L'analisi statica ha rilevato un totale di 18 code smell. Gli smell principali evidenziati sono i seguenti: 8 violazioni legate all'uso dell'annotazione `@Autowired` per l'iniezione delle dipendenze. A questo si aggiungono problemi di manutenibilità, con tre metodi segnalati per l'eccessiva complessità cognitiva, oltre a smell minori come stringhe duplicate, blocchi *try-catch* annidati e costrutti *switch* non ottimizzati.

Nelle varianti C1, C2 e C3, l'LLM si è concentrato sulla riduzione del numero di smell. L'intervento principale ha riguardato l'eliminazione delle annotazioni `@Autowired`: le varianti C1 e C2 le hanno rimosse spostando l'inizializzazione delle dipendenze all'interno di un unico costruttore. È emersa tuttavia una differenza sostanziale legata ai test: C1 include nel costruttore solo le 7 dipendenze dirette, mentre C2 estende forzatamente la firma a 15 parametri per includere le dipendenze ereditate dalla superclasse e soddisfare i test Black-Box. Questa scelta ha però comportato un costo in termini di qualità statica, introducendo in C2 un nuovo smell strutturale legato proprio all'eccessivo numero di parametri nel costruttore. Nella variante C3, invece, l'LLM ha adottato un approccio alternativo, rimuovendo le annotazioni e creando metodi *setter* dedicati per ciascuna dipendenza. Inoltre, in tutte e tre le varianti l'LLM ha inserito dichiarazioni inutili di eccezioni nelle firme dei metodi, dunque sono stati aggiunti smell differenti da quelli originari.

A differenza delle varianti precedenti, nella variante C4 l'LLM, in modo del tutto analogo a quanto osservato per la classe `ResourceLogic`, ha applicato una scomposizione architetturale decisamente più profonda. Infatti, il modello ha individuato logiche fortemente ripetute tra i vari moduli della classe. Per rimuovere la ridondanza, ha isolato il codice condiviso all'interno di nuovi metodi privati. Anche in questo secondo caso di studio si nota dunque che C4 ha subito una ristrutturazione che ottimizza la modularità complessiva della classe, mantenendone inalterate le funzionalità originali. In C4 sono rimasti 12 smell, un numero inferiore rispetto ai 18 di C0. Si è notato che i 12 smell rimanenti sono tutti ereditati dalla versione originale: a

differenza di quanto accaduto nelle altre varianti, in questo caso l'LLM non ha introdotto alcun code smell nuovo.

Anche in questo caso, sono state estratte le seguenti metriche: CBO, LOC, WMC, RFC e LCOM, per la classe originale e per le quattro varianti. I valori sono visibili in Appendice M4, [Tabella 7](#).

A differenza di quanto osservato per la classe ResourceLogic, le varianti C1, C2 e C3 mostrano un peggioramento rispetto alle metriche considerate. L'eliminazione delle annotazioni *@Autowired* in favore di costruttori, in C1 e C2, o metodi *setter*, in C3, unita all'estrazione dei blocchi *try-catch* in metodi privati, ha peggiorato la qualità del codice: la LOC ha raggiunto un massimo di 543, il WMC è salito fino a 105, mentre la LCOM è passata da 64 al valore massimo di 240, evidenziando una frammentazione della logica interna.

Per quanto riguarda C4, la LOC diminuisce, passando da 497 a 481, confermando l'effettiva rimozione delle ridondanze, ma si è evidenziato un peggioramento per l'*RFC*, che sale a 106, e per la LCOM che passa da 64 a 93, evidenziando una minore coesione interna nonostante la riduzione di righe di codice. I valori di WMC e CBO si mantengono stabili.

In conclusione, il refactoring tramite LLM riduce parzialmente gli smell, ma non li elimina del tutto. L'analisi delle metriche rivela che in nessuna delle varianti prodotte il modello è riuscito a migliorare la qualità strutturale del codice.

4. Minacce alla validità

4.1 Milestone 1

La decisione di limitare l'analisi al primo 34% delle release di Apache Syncope rappresenta un trade-off importante. Sebbene tale delimitazione sia necessaria per garantire l'affidabilità del processo di labeling, evitando il rischio di considerare "non buggy" classi che semplicemente non hanno ancora manifestato difetti, essa comporta una riduzione dei dati su cui fare una predizione.

Inoltre, l'utilizzo di *Proportion Total* per stimare l'*Injected Version* introduce un potenziale bias. Poiché la proporzione viene calcolata sull'intero storico del progetto, le stime per i bug più datati sono influenzate da informazioni future, ovvero i fix successivi, che non sono ancora disponibili al momento dell'iniezione del difetto. Tuttavia, questa scelta offre il vantaggio di non scartare ticket incompleti.

Un'ulteriore minaccia riguarda il rilevamento dei code smell tramite *PMD*. Questo garantisce uniformità nell'analisi su tutto il dataset. Tuttavia, tale analisi è intrinsecamente legata al set di regole considerato: la variazione di tali regole o l'utilizzo di uno strumento differente avrebbe potuto modificare il valore numerico di NSMELLS. Pertanto, le conclusioni riguardanti il ruolo degli smell vanno interpretate in relazione alla configurazione specifica utilizzata in questo studio.

Le stesse considerazioni fatte per *PMD* riguardano l'utilizzo dello strumento *CK* per il calcolo delle metriche strutturali. Infatti, tale analisi dipende strettamente dalle definizioni operative adottate dallo strumento. Pertanto, l'adozione di un tool differente potrebbe portare a variazioni nelle metriche estratte. È bene quindi considerare questi dati come indicazioni approssimative della complessità, piuttosto che come misure esatte.

4.2 Milestone 2

Una prima minaccia riguarda la configurazione dei classificatori scelti, ovvero IBk, NaiveBayes e RandomForest, i quali sono stati eseguiti affidandosi ai parametri di default forniti dall'ambiente Weka. Una diversa ottimizzazione degli iperparametri avrebbe potuto portare a variazioni nelle performance.

L'applicazione del Balancing tramite *Undersampling* ha comportato la rimozione di numerose istanze della classe maggioritaria. Sebbene questa tecnica eviti l'introduzione di dati sintetici, la forte riduzione dei dati di training fa perdere ai modelli informazioni utili per l'apprendimento.

Un'ulteriore minaccia riguarda l'approccio *Forward greedy*; quest'ultimo semplifica l'interpretabilità del modello grazie a un numero limitato di feature, ma il suo carattere sequenziale impedisce di valutare l'impatto globale delle variabili. Tale limite metodologico, privo di una selezione congiunta, può aver causato l'identificazione di un set di feature sub-ottimale, perdendo il contributo di combinazioni di variabili non analizzabili singolarmente.

4.3 Milestone 3

La principale minaccia alla validità riscontrata in questa Milestone riguarda la manipolazione introdotta nel Dataset B, ovvero l'aver forzato la metrica NSMELLS a 0 lasciando del tutto inalterate le restanti metriche strutturali. Nella realtà dello sviluppo software, un refactoring che elimina i code smell andrebbe inevitabilmente a modificare anche la complessità, l'accoppiamento e la coesione della classe. Generare istanze in cui gli smell vengono azzerati ma la complessità rimane identica a quando c'erano gli smell crea uno scenario artificiale che nella realtà potrebbe non esistere.

4.4 Milestone 4

La minaccia principale nell'utilizzo di Microsoft Copilot risiede nella natura probabilistica e non deterministica degli LLM. A parità di prompt iniziale e di contesto fornito, l'esecuzione del refactoring in momenti diversi produce output differenti. Questo rende i risultati ottenuti sulle varianti non perfettamente replicabili in modo deterministico. In aggiunta, i risultati sono strettamente vincolati a Copilot: l'utilizzo di altri LLM, aventi architetture e dati di addestramento diversi, potrebbe portare a prestazioni e risultati divergenti.

Inoltre, l'esperimento di refactoring è stato condotto su sole due classi specifiche del progetto. Sebbene siano state scelte per rappresentare scenari opposti, un campione così ridotto non è sufficiente per coprire la complessità del software reale. Pertanto, i risultati ottenuti non possono essere estesi con certezza ad altre classi o ad altri progetti.

Conclusioni

L'intero lavoro ha permesso dunque di rispondere alle 3 domande poste all'inizio della relazione:

- **RQ1:** Nella Milestone 2 il miglior classificatore per questo dataset è risultato essere RandomForest senza Feature Selection e senza Balancing. Il modello ha infatti garantito le performance migliori, con valori di *AUC*, *Kappa*, *Precision* e *Recall* complessivamente superiori agli altri modelli considerati.
- **RQ2:** Nella Milestone 3, l'analisi *what-if* ha evidenziato che, in uno scenario privo di code smell, si ottiene una lieve riduzione dei TP. Ciò dimostra che la feature NSMELLS è informativa, ma da sola non è sufficiente a modificare radicalmente le decisioni del modello.
- **RQ3:** Nella Milestone 4, è emerso che il refactoring automatico tramite LLM non garantisce un miglioramento complessivo della qualità del codice. Sebbene il numero di code smell diminuisca, la struttura ne risente negativamente: le metriche WMC, RFC e LCOM, che in questo caso di studio sono strettamente correlate alla difettosità, spesso peggiorano. Si evidenzia quindi un trade-off strutturale: l'intervento dell'LLM, pur riducendo gli smell (senza eliminarli del tutto), tende a frammentare la logica interna, innalzando la complessità architetturale del componente anziché ridurla.

Questo studio conferma che gli LLM rappresentano strumenti di supporto estremamente potenti, ma non ancora così precisi da agire come architetti del software autonomi. La supervisione umana rimane, ad oggi, un requisito imprescindibile per garantire che l'automazione del refactoring non sacrifichi la manutenibilità del sistema a lungo termine.

Appendice M1

Tabella 1: Riassunto delle caratteristiche complessive del dataset.

Release totali disponibili	79
Release considerate	27
Istanze dataset	17023
Istanze buggy	1145
Istanze non buggy	15878
Percentuale istanze buggy	6.73%
Istanze con almeno uno smell	10665
Istanze senza smell	6358
Percentuale istanze con smell	62.65%

Tabella 2: Distribuzione delle classi.

Release	# Classi	# Classi Buggy	% Classi Buggy	# Classi con smell	% Classi con smell
1	466	21	4.51%	307	65.88%
2	466	8	1.72%	306	65.67%
3	466	29	6.22%	305	65.45%
4	467	18	3.85%	306	65.52%
5	467	13	2.78%	306	65.52%
6	469	109	23.24%	309	65.88%
7	476	66	13.87%	313	65.76%
8	476	48	10.08%	313	65.76%
9	475	54	11.37%	312	65.68%
10	621	87	14.01%	381	61.35%
11	622	61	9.81%	381	61.25%
12	625	54	8.64%	384	61.44%
13	629	84	13.35%	386	61.37%
14	631	66	10.46%	385	61.01%
15	643	86	13.37%	397	61.74%
16	643	16	2.49%	399	62.05%
17	645	39	6.05%	400	62.02%
18	645	24	3.72%	400	62.02%
19	781	48	6.15%	484	61.97%
20	764	26	3.40%	472	61.78%
21	787	22	2.80%	485	61.63%
22	788	21	2.66%	484	61.42%
23	789	18	2.28%	488	61.85%
24	790	30	3.80%	488	61.77%
25	796	35	4.40%	490	61.56%
26	797	32	4.02%	491	61.61%
27	799	30	3.75%	493	61.70%

Appendice M2

Tabella 3: Risultati delle metriche sui classificatori presi in considerazione.

Classificatore	Feature Selection	Balancing	Precision	Recall	AUC	Kappa
RandomForest	No	No	0.66	0.61	0.92	0.61
NaiveBayes	No	No	0.19	0.29	0.71	0.15
Ibk	No	No	0.53	0.52	0.76	0.49
RandomForest	Yes	No	0.68	0.59	0.92	0.61
NaiveBayes	Yes	No	0.21	0.29	0.71	0.18
Ibk	Yes	No	0.48	0.45	0.72	0.43
RandomForest	No	Yes	0.52	0.56	0.90	0.50
NaiveBayes	No	Yes	0.18	0.31	0.71	0.15
Ibk	No	Yes	0.16	0.80	0.75	0.18
RandomForest	Yes	Yes	0.48	0.55	0.89	0.47
NaiveBayes	Yes	Yes	0.20	0.29	0.71	0.17
Ibk	Yes	Yes	0.15	0.77	0.73	0.16

N.B: Il classificatore evidenziato in rosso rappresenta il *Best Classifier* per questo dataset.

Appendice M3

Tabella 4: Risultati delle predizioni sui dataset della what-if analysis usando RandomForest.

Dataset	Tot istanze	Buggy Reali	Buggy predette	TP	FP	TN	FN
A	17023	1145	1146	1143	3	15875	2
B+	10665	928	928	926	2	9735	2
B	10665	928	939	911	28	9709	17
C	6358	217	218	217	1	6140	0

Appendice M4

Prompt Refactoring:

"You are an expert Java developer. I want to improve the maintainability of the attached C_0 class: <path_class>. Create C_X without changing C_0 functionality and by removing the following smells that you can find in the photo attached to this prompt. Do not include in C_X changes different from what I asked. In fact, consider that C_X should replace C_0 and work with the other components of the system as C_0 currently does. This is an important request; take all the time you need to provide a complete and accurate answer to this request. Class source code: <source_code>."

Tabella 5: Correlazione di Spearman delle feature statiche selezionate.

Feature statica	Spearman ρ	p-value
CBO	0.205	< 0.001
RFC	0.190	< 0.001
LOC	0.189	< 0.001
WMC	0.187	< 0.001
NSMELLS	0.144	< 0.001
LCOM	0.098	<0.001

Tabella 6: Valori delle metriche su tutte le varianti per ResourceLogic.

Metriche	C0	C1	C2	C3	C4
LOC	207	209	209	211	204
WMC	37	37	37	36	39
RFC	72	72	72	72	76
CBO	36	36	36	36	36
LCOM	0	0	0	0	19
NSMELLS	4	0	0	2	2

Tabella 7: Valori delle metriche su tutte le varianti per DefaultRealmPullResultHandler.

Metriche	C0	C1	C2	C3	C4
LOC	497	528	526	543	481
WMC	95	100	97	105	95
RFC	103	109	106	108	106
CBO	28	28	36	28	28
LCOM	64	143	98	240	93
NSMELLS	18	2	1	3	12